
Plate-Local River Generation: Deterministic, Terrain-Aware, Downhill-Flowing Rivers for Infinite Procedural Worlds

Michael K. Davis III, Independent Researcher, USA
mikedavismk3@icloud.com

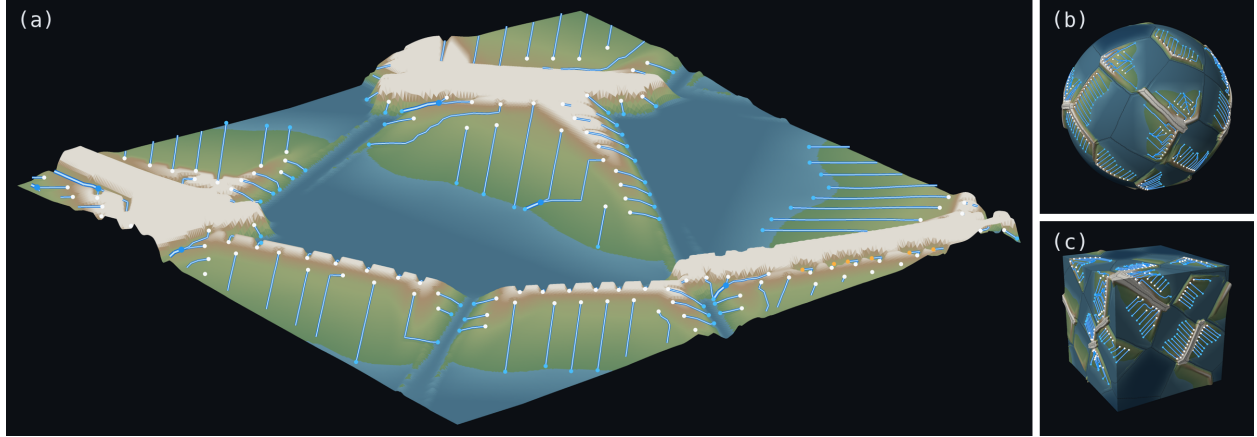


Figure 1: Plate-local river networks generated on demand. (a) A regional view of the flat infinite-world implementation evaluated in Section 4.1, in which ten plate domains independently generate terrain-aware, downhill-flowing river networks without global preprocessing. (b) Sphere and (c) cube planets from the finite-world toy implementations, each with 40 plates; rivers are cached for the plates within 55° of the view direction, each built the first time its plate faces the camera. River geometry is depth-tested against the terrain, hiding channels occluded by ridges. White dots indicate source nodes, light blue dots sea outlets, darker blue dots confluences, and yellow dots lakes at local minima.

Abstract

Existing procedural terrain generation methods produce coherent, downhill-flowing river networks only on fixed domains, whereas approaches that extend to infinite worlds typically sacrifice network coherence or downhill-flow guarantees. This work presents a deterministic, terrain-aware, downhill-flowing river algorithm for infinite procedural worlds. Tectonic plate seeds define both continuous terrain fields and corresponding bounded Voronoi-like plate domains, so localized river preprocessing can operate directly on the geometric domain without sampling terrain fields for the purpose of locating plate boundaries. Within each plate, the terrain field is sampled on a low-resolution grid, and rivers are traced downhill from highland source nodes to coastlines, local minima, or existing confluences. River heights are then

monotonically adjusted along each path so that every stored segment descends. The resulting river network is generated and cached lazily per plate, so each plate is processed independently of its neighbors. During full-resolution terrain sampling, the distance to the cached river network is computed exactly, and river height is interpolated along nearby segments and blended with the locally evaluated terrain on demand, without caching the blended result itself. Because the warp of the plate borders is bounded below the river border margin, this guarantees downhill flow along every river channel of the final terrain. Together, these design choices make terrain-aware river generation practical for interactive procedural worlds by supporting efficient on-demand evaluation, as demonstrated by the performance analysis presented in this work.

1 Introduction

Procedural, deterministic, locally generatable terrain is well suited to large worlds because the terrain can be generated in independent chunks that remain seamless with neighboring chunks. However, large-scale features such as river networks may depend on terrain relationships that extend far beyond the chunk currently being generated.

This work resolves that tension by using tectonic plates as an intermediate preprocessing domain—large enough to contain a plate’s regional drainage structure, but small enough to generate and cache independently of the rest of the world. A plate’s river network can be built directly from a geometric representation of the plate’s domain, which itself does not require sampling the terrain fields to find its borders. Within this geometric domain, a low-resolution sample of the terrain fields is taken to guide downhill river generation. Because rivers are generated independently for each plate, they do not cross plate boundaries. This limitation is mitigated by constructing the continuous terrain fields so that oceans, plateaus, and mountain ranges naturally discourage cross-boundary drainage.

The key contributions of this work are:

- dual plate representation for terrain sampling and river preprocessing, described in Sections 3.2–3.4;
- lazy local river generation and caching, described in Section 3.7.

2 Related Works

Related works for fixed domains generally fall into two broad categories: systems that construct rivers first, then the terrain is constructed around them; and systems that construct the terrain first, then the rivers are constructed along this terrain. Learned terrain generation offers a novel approach to world generation in general, and can be extended to infinite domains [13].

2.1 River-First Generation

River-first approaches generate river network structures before the surrounding terrain [10–12]. By treating rivers as the primary organizing structure, these

methods guarantee coherent, downhill-flowing rivers while allowing explicit control over river placement, connectivity, and slope. However, because terrain is constructed around a predefined river network, large-scale geological processes such as tectonic uplift and mountain building are secondary to river constraints. Furthermore, these approaches require preprocessing over a fixed domain and are not readily extendable to infinite procedural worlds.

2.2 Terrain-First Generation

Terrain-first approaches first generate terrain before deriving river networks from the resulting topology. Batbold and Ziegelmeier [9] demonstrate that coherent river networks can be derived from procedurally generated terrain using graph-based flow routing. The present work builds on this terrain-first philosophy with river generation derived from a low-resolution sample of terrain fields, which is localized to plate domains. This enables river generation without preprocessing the entire domain.

2.3 Learned Generation

InfiniteDiffusion [13] resolves the infinite-domain limitation of both river-first and terrain-first methods. Current limitations of diffusion-based terrain generators include limited explicit control over large-scale terrain features, such as continent placement, mountain range orientation, or coastline shape and no guarantee of coherent, downhill-flowing river networks. These limitations are not necessarily fundamental to diffusion models, but do remain unaddressed.

3 Plate-Local River Generation

The following subsections define deterministic plate seeds and their dual pointwise and geometric representations, then describe plate-local terrain sampling, river tracing, caching, and on-demand composition. Together, these stages preserve terrain-aware downhill drainage without a global preprocessing pass. For applications to finite, closed-shaped worlds such as planet domains, we recommend slight modifications to this system, which we provide in this section alongside the infinite application.

Figure 2 summarizes this pipeline, separating the per-

plate cache build from on-demand terrain evaluation. Algorithms 1 and 2 detail the complete procedures for building plate-local river caches and evaluating terrain on demand.

Algorithm 1. *Plate-Local River Cache Build*

Require: Plate i , seed positions $\{\mathbf{c}_j\}$, surface $\mathcal{S}$
Ensure: River network $\mathcal{R}_i$

Geometric plate domain (Section 3.4)

- 1: Initialize bounding volume B_i around $\mathbf{c}_i$
- 2: **for** each neighboring plate $j \in \mathcal{N}_i$ **do**
- 3: $\mathbf{m}_{ij} \leftarrow \frac{1}{2}(\mathbf{c}_i + \mathbf{c}_j)$; $\mathbf{n}_{ij} \leftarrow \mathbf{c}_j - \mathbf{c}_i$
- 4: Clip B_i by $(\mathbf{p} - \mathbf{m}_{ij}) \cdot \mathbf{n}_{ij} \leq 0$
- 5: **end for**
- 6: $\Omega_i^{\mathcal{S}} \leftarrow B_i \cap \mathcal{S}$
- Low-resolution plate grid (Section 3.5)*
- 7: **for** each (u, v) with $\mathbf{x}_{uv} \in \Omega_i^{\mathcal{S}}$ **do**
- 8: $h_{uv} \leftarrow H_{\text{plan}}(\mathbf{x}_{uv})$
- 9: Mark (u, v) border-safe and/or lake-safe from $\text{dist}(\mathbf{x}_{uv}, \partial\Omega_i^{\mathcal{S}})$
- 10: **end for**
- River node and segment generation (Section 3.6)*
- 11: $\mathcal{R}_i \leftarrow \emptyset$
- 12: **for** each border-safe source candidate, in descending height order **do**
- 13: **if** K paths are accepted **then**
- 14: **break**
- 15: **end if**
- 16: **if** the candidate is within $s_{\min}$ of an existing river **then**
- 17: **continue**
- 18: **end if**
- 19: **repeat**
- 20: **if** at sea level **then**
- 21: Mark a sea outlet; **break**
- 22: **end if**
- 23: **if** a lower existing node is within reach **then**
- 24: Join it as a confluence; **break**
- 25: **end if**
- 26: Step to the lowest border-safe, non-crossing lower sample
- 27: **until** no lower sample exists or $n_{\max}$ steps are taken
- 28: At a dead end: mark a lake if lake-safe, else discard the path
- 29: Add the accepted path's segments to $\mathcal{R}_i$
- 30: **end for**
- 31: Monotonically adjust river heights along each path
- 32: Assign Strahler orders; cache grid, nodes, segments, and paths

Algorithm 2. *On-Demand Terrain Sample*

Require: Position $\mathbf{x}$, seed positions $\{\mathbf{c}_j\}$, river cache
Ensure: Final height $H_{\text{final}}(\mathbf{x})$

Plate assignment (Section 3.3)

- 1: $\tilde{\mathbf{x}} \leftarrow s_p \mathbf{x} + \mathbf{w}(\mathbf{x})$
- 2: $i \leftarrow \arg \min_{j \in \mathcal{A}} \|\tilde{\mathbf{x}} - \mathbf{c}_j\|^2$ $\triangleright$ Owning plate; local 3×3 for 2D or $3 \times 3 \times 3$ for 3D lattice search if $\mathcal{A} = \mathbb{Z}^d$
- Cache lookup and lazy build (Section 3.7)*
- 3: **if** the cache for plate i is absent **then**
- 4: Execute Algorithm 1 for plate i
- 5: **end if**
- Pointwise terrain fields (Section 3.3)*
- 6: Evaluate C , W_{land} , W_{mountain} , W_{plateau} , and W_{island} at $\mathbf{x}$
- 7: Compute $H_{\text{pre}}(\mathbf{x})$ from the weighted fields
- River query (Section 3.7)*
- 8: **if** $\mathcal{R}_i = \emptyset$ **then**
- 9: **return** $H_{\text{pre}}(\mathbf{x})$
- 10: **end if**
- 11: $\delta_{\text{river}}(\mathbf{x}) \leftarrow \min_{s \in \mathcal{R}_i} \delta_s(\mathbf{x})$ $\triangleright$ Scan all segments of plate i ; lake disks included
- 12: Blend $h_s(\mathbf{x})$ over nearby s into $H_{\text{river}}(\mathbf{x})$
- Terrain composition (Section 3.8)*
- 13: $\lambda_{\text{river}}(\mathbf{x}) \leftarrow \left[\min \left(1, \frac{[\delta_{\text{river}}(\mathbf{x})]_+}{\omega_{\text{river}}} \right) \right]^{\alpha_{\text{river}}}$
- 14: $H_{\text{final}}(\mathbf{x}) \leftarrow \lambda_{\text{river}}(\mathbf{x}) H_{\text{pre}}(\mathbf{x}) + (1 - \lambda_{\text{river}}(\mathbf{x})) H_{\text{river}}(\mathbf{x})$
- 15: **return** $H_{\text{final}}(\mathbf{x})$

3.1 Deterministic Tectonic Plate Positions

Each tectonic plate is defined by a deterministic seed position. The same construction can be used for finite and infinite worlds: finite worlds use a bounded set of plate identifiers, while infinite worlds generate plate identifiers from lattice cells on demand. In both cases, terrain field samples are assigned to the nearest deterministic seed position, producing Voronoi-like plate regions [1, 2].

For an infinite world, plate seeds may be generated as one deterministic point per integer lattice cell, following the same nearest-feature-point structure used by cellular texture bases [6],

$$\mathbf{c}_i = \mathbf{i} + \frac{1}{2} \mathbf{1} + r \boldsymbol{\eta}(\mathbf{i}), \quad \mathbf{i} \in \mathbb{Z}^d, \quad (1)$$

where i is the plate identifier associated with integer lattice-cell coordinate $\mathbf{i}$, $\boldsymbol{\eta}(\mathbf{i}) \in [-1, 1]^d$ is a deterministic per-axis hash offset, and r bounds the jitter in cell widths. Seeds are expressed in *plate space*, in which world coordinates are multiplied by the plate scale s_p (Equation (5)), so one lattice cell spans $1/s_p$

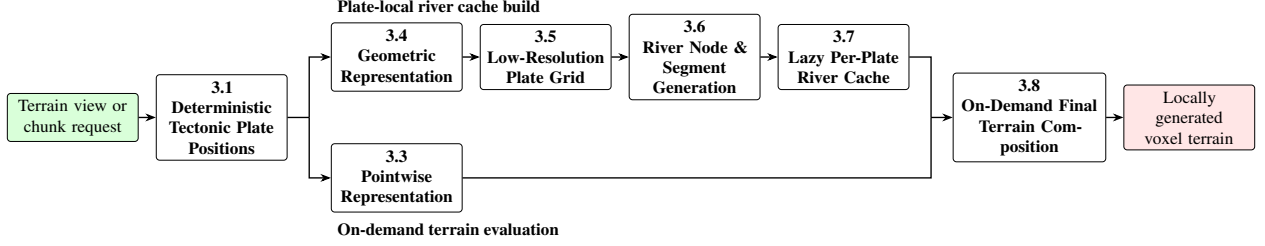


Figure 2: Pipeline overview, generalized for various world types—infinite or finite, and landscape structures—voxels or mesh.

world units. For this present implementation, $\eta(\mathbf{i})$ remains an offset on the 2D plane. Considering an additional offset in 3D space will result in continental landmasses that are less uniform in size, which may be preferred.

For a finite world, with a closed geometry and N plates, a golden-angle sequence gives approximately even seed directions [3, 4]:

$$\theta_i = i\pi(3 - \sqrt{5}), \quad y_i = 1 - \frac{2i+1}{N}, \quad r_i = \sqrt{1 - y_i^2}, \quad (2)$$

$$\mathbf{q}_i = (r_i \cos \theta_i, y_i, r_i \sin \theta_i), \quad (3)$$

$$\mathbf{c}_i = s_p \Pi_{\mathcal{S}}(\mathbf{q}_i) + \eta_i, \quad 0 \leq i < N, \quad (4)$$

Here $\Pi_{\mathcal{S}}$ projects the unit direction onto the chosen finite world surface, and η_i is a deterministic jitter applied to the finite plate center.

3.2 Dual Tectonic Plate Representation

The tectonic seed positions are used in two forms: a *pointwise representation*, defined by continuous coordinate-based fields for terrain evaluation, and a *geometric representation*, defined by a bounded non-rasterized domain for regional river preprocessing. The same seed position is used to determine both representations for that given plate.

Figure 3 shows both representations derived from the same plate seed.

3.3 Pointwise Representation

During chunk generation, terrain is evaluated directly from world-space coordinates by comparing plate seed positions and deriving continuous scalar and vector weight fields from plate ownership, plate type, border distance, and drift direction. These weight fields control land, mountain, plateau, and island

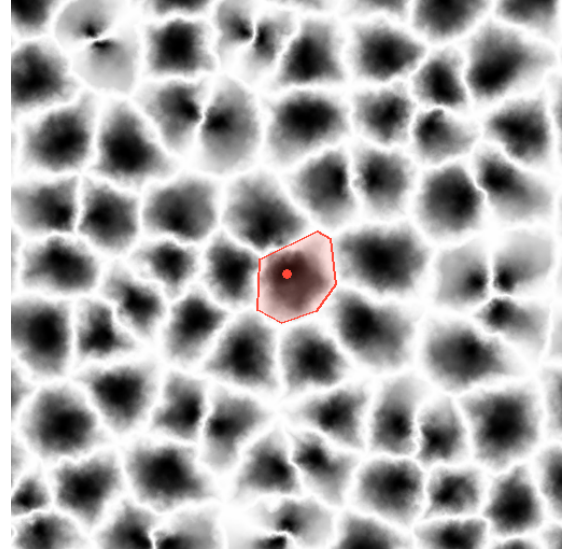


Figure 3: Dual plate representations. A continuous terrain field (grayscale) and a corresponding geometric plate domain (red) are independently derived from the same tectonic plate seed.

chain placement. Noise warping and surface roughness are applied to produce more organic continent shapes [5].

The primary purposes of the pointwise representation are to generate terrain features that appear natural, and to generate terrain features with natural barriers, such as mountains, plateaus, and oceans, at plate borders, providing a natural motive for localizing rivers to each plate and prohibiting plate boundary crossings. Otherwise, the pointwise representation is not essential to this system, and the continuous terrain fields can be constructed to disregard the plate boundaries entirely.

As a qualitative illustration, Figure 4 shows how several plate-tessellation parameters shape the terrain,

using renders from an earlier prototype of the pointwise representation.

Pointwise Plate. For a world-space point $\mathbf{x}$, the pointwise plate fields are evaluated in warped plate space,

$$\tilde{\mathbf{x}} = s_p \mathbf{x} + \mathbf{w}(\mathbf{x}), \quad (5)$$

where $\mathbf{w}(\mathbf{x}) = a_w(n_1(\mathbf{x}), n_2(\mathbf{x}))$ is a deterministic noise warp built from two normalized fractal noise fields $n_1, n_2 \in [-1, 1]$ with amplitude a_w in plate cells, so $\|\mathbf{w}\| \leq \sqrt{2} a_w$. Note that $\mathbf{w}$ cannot be represented in the geometric representation of the same plate. Given deterministic plate centers $\mathbf{c}_i$, the owning plate is the nearest center in this space:

$$i = \arg \min_{j \in \mathcal{A}} \|\tilde{\mathbf{x}} - \mathbf{c}_j\|^2. \quad (6)$$

Here $\mathcal{A}$ is the set of possible plate identifiers. For a finite world, $\mathcal{A} = \{1, \dots, N\}$ and each identifier names one generated plate seed. For an infinite world, $\mathcal{A} = \mathbb{Z}^d$ and each identifier is an integer lattice-cell coordinate that generates one seed on demand. Rather than searching all of $\mathbb{Z}^d$, the implementation restricts this arg min to the 3×3 for 2D or $3 \times 3 \times 3$ for 3D neighborhood of lattice cells surrounding $\tilde{\mathbf{x}}$'s own cell, following the same bounded-jitter neighborhood search used for cellular texture bases [6]. This restriction is exact, independent of the warp $\mathbf{w}$, provided the per-axis hash offset is bounded to at most r cell-widths from the cell center: for a query point at a corner shared by four cells, each of those four seeds can be pushed no farther than $(0.5 + r)\sqrt{2}$ from the corner, while a seed two cells away along either axis can be pulled as close as $\sqrt{(1.5 - r)^2 + (r - 0.5)^2}$. These two distances are equal at $r = 1/3$, so bounding the hash offset to $r \leq 1/3$ guarantees the neighborhood always contains the true nearest seed; the implementation enforces $r = 1/3$. Plate fields are controlled by the distance margin between the owning plate and a neighboring plate:

$$d_j(\mathbf{x}) = \|\tilde{\mathbf{x}} - \mathbf{c}_j\| - \|\tilde{\mathbf{x}} - \mathbf{c}_i\|, \quad j \neq i. \quad (7)$$

Small values of d_j indicate proximity to the boundary between the owning plate i and neighbor plate j . When a terrain feature depends on the nearest eligible

neighboring plate, the margin is minimized over a selected neighbor set $\mathcal{N}$:

$$d_{\mathcal{N}}(\mathbf{x}) = \min_{j \in \mathcal{N}} d_j(\mathbf{x}). \quad (8)$$

Plate Drift. To mimic the effects of tectonic plate drift, where land, mountains, and plateaus are largest near colliding plate borders, we define plate drift vector and scalar fields to guide the magnitudes of subsequent weight fields.

Each plate identifier is assigned a deterministic drift axis $\mathbf{a}_i$ by projecting a random vector $\mathbf{r}_i$ into the tangent plane of the terrain domain. For a non-flat world, such as a finite planet with a closed geometry, the plate center direction vector $\mathbf{c}_i$ from the world origin provides a stable reference normal:

$$\mathbf{a}_i = \frac{\mathbf{c}_i \times \mathbf{r}_i}{\|\mathbf{c}_i \times \mathbf{r}_i\|}. \quad (9)$$

For a flat world, such as an infinite one, the constant reference normal $\mathbf{n}_0$ can be used instead:

$$\mathbf{a}_i = \frac{\mathbf{n}_0 \times \mathbf{r}_i}{\|\mathbf{n}_0 \times \mathbf{r}_i\|}. \quad (10)$$

The drift direction at the terrain sample is then

$$\ell(\mathbf{x}) = \frac{\mathbf{a}_i \times \mathbf{n}(\mathbf{x})}{\|\mathbf{a}_i \times \mathbf{n}(\mathbf{x})\|}, \quad (11)$$

where $\mathbf{n}(\mathbf{x})$ is the local surface normal, and $\ell(\mathbf{x})$ is the local tangent drift direction used at the terrain sample. For a cube-shaped planet, the implementation uses the radial direction $\mathbf{x}/\|\mathbf{x}\|$ in place of the true face normal, which keeps ℓ continuous across cube edges. Note that for a flat world, this greatly simplifies to a constant per-plate direction $\ell(\mathbf{x}) = \mathbf{r}_i/\|\mathbf{r}_i\|$ with $\mathbf{r}_i \in \mathbb{R}^2$.

The scalar drift field, $\rho(\mathbf{x})$, measures signed progress along the local drift direction:

$$\rho(\mathbf{x}) = (\tilde{\mathbf{x}} - \mathbf{c}_i) \cdot \ell(\mathbf{x}). \quad (12)$$

Weights. Land, island, and mountain feature weights use the same normalized beta-shaped border profile. For feature k with amplitude A_k , width

ω_k , interior exponent p_k , border exponent q_k , and plate-coverage parameter γ_k ,

$$C_k = \left[\left(\frac{p_k}{p_k + q_k} \right)^{p_k} \left(\frac{q_k}{p_k + q_k} \right)^{q_k} \right]^{-1}, \quad (13)$$

$$D_k(\mathbf{x}) = \max \left(0, \frac{\rho(\mathbf{x}) + 2\gamma_k - 1}{2\gamma_k} \right), \quad (14)$$

$$z_k(\mathbf{x}) = \frac{d_{\mathcal{N}_k}(\mathbf{x})}{\omega_k}, \quad (15)$$

$$W_k(\mathbf{x}) = \begin{cases} A_k C_k D_k(\mathbf{x}) z_k(\mathbf{x})^{p_k} \cdot (1 - z_k(\mathbf{x}))^{q_k}, & 0 \leq z_k(\mathbf{x}) \leq 1, \\ 0, & \text{otherwise.} \end{cases} \quad (16)$$

The coefficient C_k normalizes the profile before amplitude scaling, and D_k gates the feature by the drift field.

Plateaus are treated separately because, in this work, we define them as highlands arising from converging continental plates rather than a single boundary-distance profile. For a nearby continental pair (i, j) , let $\hat{\mathbf{u}}_{ij}(\mathbf{x})$ denote the unit separation direction from plate i to plate j after projection into the local tangent plane. The closing response is measured by projecting each plate's tangent drift onto this direction:

$$\kappa_{i \rightarrow j}(\mathbf{x}) = \begin{cases} (\mathbf{a}_i \times \mathbf{n}(\mathbf{x})) \cdot \hat{\mathbf{u}}_{ij}(\mathbf{x}), & \text{if positive,} \\ 0, & \text{if zero or negative,} \end{cases} \quad (17)$$

$$\kappa_{j \rightarrow i}(\mathbf{x}) = \begin{cases} -(\mathbf{a}_j \times \mathbf{n}(\mathbf{x})) \cdot \hat{\mathbf{u}}_{ij}(\mathbf{x}), & \text{if positive,} \\ 0, & \text{if zero or negative.} \end{cases} \quad (18)$$

These detect colliding plates. The weight itself is constructed as,

$$R_{\text{plateau}}(\mathbf{x}) = O(\mathbf{x}) \max_{(i,j) \in \mathcal{C}} B_{ij}(\mathbf{x}) N_i(\mathbf{x}) \cdot \kappa_{i \rightarrow j}(\mathbf{x}) \kappa_{j \rightarrow i}(\mathbf{x}) \cdot S\left(\frac{\kappa_{i \rightarrow j}(\mathbf{x})}{\eta}\right) S\left(\frac{\kappa_{j \rightarrow i}(\mathbf{x})}{\eta}\right), \quad (19)$$

$$W_{\text{plateau}}(\mathbf{x}) = A_{\text{plateau}} S(g_{\text{plateau}} R_{\text{plateau}}(\mathbf{x})) \cdot (A_{\text{land}} - W_{\text{land}}(\mathbf{x})) \cdot (A_{\text{mountain}} - W_{\text{mountain}}(\mathbf{x})). \quad (20)$$

Here $\mathcal{C}$ is the set of continental plate pairs among the plates considered at $\mathbf{x}$ (the 3×3 lattice neighborhood for an infinite world), S is a smoothstep function, and

η is the closing fade width. With bridge width ω_b , $B_{ij}(\mathbf{x}) = 1 - S((\|\tilde{\mathbf{x}} - \mathbf{c}_j\| - \|\tilde{\mathbf{x}} - \mathbf{c}_i\|)/\omega_b)$ fades with the border margin between plates i and j at $\mathbf{x}$, $N_i(\mathbf{x})$ applies the same fade to the margin between plate i and the owning plate, and $O(\mathbf{x}) = S(d_{\text{ocean}}(\mathbf{x})/\omega_b)$ suppresses plateau formation near oceanic boundaries, where d_{ocean} is the margin to the nearest oceanic plate. The gain g_{plateau} controls the plateau response ($g_{\text{plateau}} = 4$ in the implementation). The final plateau weight W_{plateau} is an amplitude-scaled smoothstep response to the closing response R_{plateau} , suppressed where the land and mountain fields already dominate; the implementation clamps both suppression factors at zero.

Island chains are further confined to the leading edge of each plate and removed beneath plateaus:

$$W_{\text{island}} \leftarrow W_{\text{island}} \max \left(0, 1 + h_{\text{sea}} - \frac{\rho(\mathbf{x}) - \rho_0}{L} \right) \cdot (1 - \min(1, W_{\text{plateau}})), \quad (21)$$

where h_{sea} is the sea level, $\rho_0 = 1 - 2\gamma_{\text{land}} + h_{\text{sea}}$, and $L = 2(\gamma_{\text{island}} - \gamma_{\text{land}})$. Oceanic plates receive none of the land, island, mountain, or plateau weights.

Pre-River Terrain Height Composition. The pre-river terrain height $H_{\text{pre}}(\mathbf{x})$ is the sum of a crust field and the land, island, mountain, and plateau weight fields:

$$H_{\text{pre}}(\mathbf{x}) = C(\mathbf{x}) + W_{\text{land}}(\mathbf{x}) + W_{\text{mountain}}(\mathbf{x}) + W_{\text{plateau}}(\mathbf{x}) + n_{\text{island}}(\mathbf{x}) W_{\text{island}}(\mathbf{x}), \quad (22)$$

where $n_{\text{island}}(\mathbf{x}) \in [0, 1]$ is low-frequency noise that breaks island chains into individual islands. The crust field sets the height floor that distinguishes continental from oceanic plates. It is the constant h_{cont} on continental plates, while on oceanic plates it rises toward h_{cont} near continental borders, forming a continental shelf:

$$C(\mathbf{x}) = h_{\text{ocean}} + (h_{\text{cont}} - h_{\text{ocean}}) \left[1 - \frac{d_{\text{cont}}(\mathbf{x})}{\omega_{\text{shelf}}} \right]_+^{p_{\text{shelf}}}, \quad (23)$$

where d_{cont} is the margin to the nearest continental plate, ω_{shelf} is the shelf width, p_{shelf} is the shelf exponent, and $[\cdot]_+ = \max(0, \cdot)$.

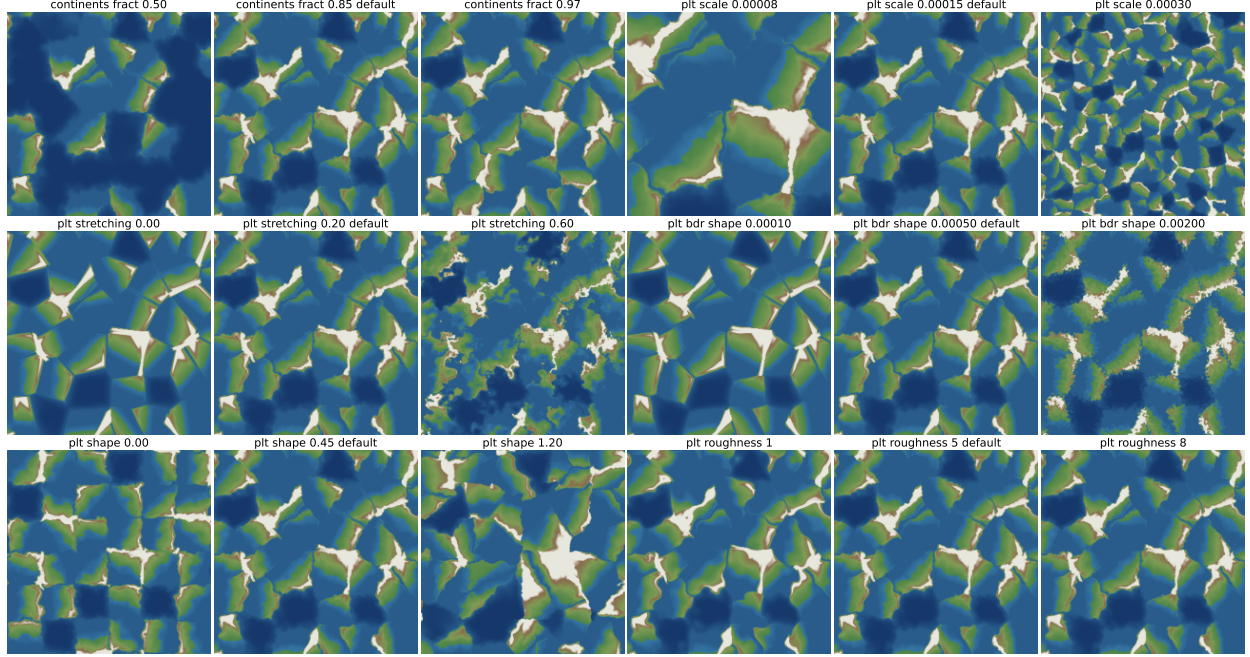


Figure 4: Illustrative top-down height-field renders from an earlier prototype of the pointwise terrain, showing how individual plate-tessellation parameters change the terrain. Each panel varies one parameter (low, prototype default, high) for one fixed seed, with all other parameters at the prototype’s defaults; panel labels give the prototype’s parameter names and values. The prototype’s terrain and defaults differ from the implementation evaluated in the results section, so these renders are not comparable to Figure 1. Row 1: oceanic/continental threshold, then plate scale s_p . Row 2: warp amplitude, then warp spatial frequency. Row 3: plate-seed jitter (zero jitter collapses to a regular lattice), then warp octave count.

3.4 Geometric Representation

The entire plate domain can be constructed geometrically from nearby seed positions without terrain field sampling. Because the noise warping applied to the weight fields cannot be reproduced geometrically, this domain is a straight-bordered approximation of the true plate footprint—close enough for river generation purposes.

The geometric plate domain is initialized as a bounded volume B_i around the plate center and then clipped by the bisector planes between $\mathbf{c}_i$ and $\mathbf{c}_j$:

$$\Omega_i = B_i \cap \bigcap_{j \neq i} \{ \mathbf{p} : (\mathbf{p} - \mathbf{m}_{ij}) \cdot \mathbf{n}_{ij} \leq 0 \}, \quad (24)$$

where

$$\mathbf{m}_{ij} = \frac{1}{2} (\mathbf{c}_i + \mathbf{c}_j), \quad (25)$$

$$\mathbf{n}_{ij} = \mathbf{c}_j - \mathbf{c}_i. \quad (26)$$

In practice the intersection runs over a finite neighbor set $j \in \mathcal{N}_i$. For an infinite world, the implementation

uses the eight lattice cells surrounding plate i , and B_i is the axis-aligned rectangle enclosing these nine seeds, padded by two cell widths. For a finite world, $\mathcal{N}_i$ contains every other plate. For a world surface $\mathcal{S}$, the plate-local preprocessing domain is the surface cross section of this clipped volume,

$$\Omega_i^{\mathcal{S}} = \Omega_i \cap \mathcal{S}. \quad (27)$$

For a flat world, $\Omega_i^{\mathcal{S}}$ is a single convex polygon in plate space. For a cube-shaped planet, Ω_i is a clipped polyhedron and $\Omega_i^{\mathcal{S}}$ is the union of up to six convex polygons, one for each cube face the plate touches, each trimmed by the adjacent face planes.

In the implementation, each half-plane (or half-space) clip is performed in the manner of the Sutherland–Hodgman algorithm [7]. Recall that this geometric representation is not identical to the pointwise representation due to the warp term $\mathbf{w}$ in Equation (5). The two borders differ by at most $\sqrt{2}a_w/s_p$ world units, which Section 3.6 uses to keep rivers inside their pointwise plate.

3.5 Low-Resolution Plate Grid

The river generator samples pre-river terrain using the weight fields on a low-resolution grid restricted to the geometric representation domain, providing approximate terrain elevation and slope for river path planning. Using a low-resolution terrain sample makes plate-local river preprocessing computationally practical by significantly reducing computation time and memory requirements, while still retaining a terrain-first generation philosophy.

For an infinite world, the implementation lays an $n_g \times n_g$ grid over the axis-aligned bounding rectangle of $\Omega_i^{\mathcal{S}}$ and keeps the samples that fall inside the polygon. For a cube-shaped planet, the grid covers each cube face that the plate touches and is cropped to that face's polygon. Each accepted grid sample (u, v) on plate i lies in the geometric surface domain and stores an attenuated planning height:

$$\begin{aligned} \mathbf{x}_{uv} \in \Omega_i^{\mathcal{S}}, \quad h_{uv} &= H_{\text{plan}}(\mathbf{x}_{uv}), \\ H_{\text{plan}} &= C + a_{\text{land}} W_{\text{land}} \\ &\quad + a_{\text{relief}} (W_{\text{mountain}} + W_{\text{plateau}} + W_{\text{island}}), \end{aligned} \quad (28)$$

where $a_{\text{land}} \leq 1$ and $a_{\text{relief}} < 1$ are pre-river attenuation factors ($a_{\text{land}} = 1$ and $a_{\text{relief}} = 0.35$ in the implementation), and the island noise of Equation (22) is omitted. Because H_{plan} flattens mountains and plateaus relative to H_{pre} , the river elevations derived from it generally lie below the surrounding terrain surface, which produces valley-like depressions after composition.

Each sample also records its distance to the domain border $\partial\Omega_i^{\mathcal{S}}$. A sample is *border-safe* if this distance is at least the border margin ω_{border} , and *lake-safe* if it is at least $\omega_{\text{border}} + r_{\text{lake}} + \omega_{\text{river}}$, where r_{lake} is the lake radius (Section 3.7).

3.6 River Node & Segment Generation

Sources. Candidate sources are border-safe grid samples with $h_{uv} \geq \max(h_{\text{src}}, h_{\text{sea}})$. Candidates are visited in descending height order. A candidate is skipped if it lies within s_{min} grid cells of any river already accepted on the plate, including that river's source; otherwise it is traced immediately. Selection stops once K paths have been accepted. Because this ordering depends only on the grid heights, source se-

lection is deterministic. Keeping sources away from existing rivers means that every path begins at a new node, so paths share nodes only at confluences, and no new segment can overlap an existing one.

Tracing. Each source is traced downhill one step at a time. From the current sample, the next sample is chosen within a $(2k+1) \times (2k+1)$ grid window: it must be valid, border-safe, and lower by at least a minimum drop $\varepsilon > 0$, and the new segment must not cross any existing segment. Among these, the sample minimizing $h_{uv} + \beta \|\Delta_{uv}\|^2$ is selected, where the small weight β favors shorter steps. The trace terminates when one of the following occurs:

- the current sample is at or below sea level, producing a *sea outlet*;
- an existing non-source node within k cells is lower by at least ε and reachable without crossing, or the chosen step lands on an existing node, producing a *confluence*;
- no admissible lower sample exists, producing a *lake* at a local minimum, provided that sample is lake-safe;
- the step budget n_{max} is exhausted.

If a dead end is not lake-safe, or if the only descent available would enter the border band, the entire path is discarded rather than truncated. Rivers therefore never drain into a plate border.

Monotone River Heights. The raw heights h strictly decrease along every accepted path by construction. Each path $(n_0, \dots, n_m)$ is then assigned river heights H by an affine remapping between its source and outlet. The outlet height H_m is its raw height, or, if the path ends at a confluence, the already-assigned river height of the node it joins. The source height is $H_0 = \max(h_0 - \Delta_{\text{drop}}, H_m + \varepsilon)$, and

$$H_k = H_m + (H_0 - H_m) \frac{h_k - h_m}{h_0 - h_m}, \quad 0 \leq k \leq m. \quad (29)$$

Since h_k is strictly decreasing in k , so is H_k . Paths are processed in generation order, so each tributary is

fitted to the river height of the trunk it joins. Because paths share nodes only at confluences, no later refit changes an earlier path’s heights, and heights agree at every confluence. Every stored segment is therefore strictly downhill. Finally, Strahler stream orders [8] are computed over the finished network and stored per segment: source tributaries receive order one, and the order increases where two equal-order tributaries meet.

Note that the algorithm for placing nodes and segments is arbitrary and can be modified or replaced without affecting the rest of the pipeline. However, to avoid blending discontinuities during final terrain composition, nodes and segments should not be placed near plate borders. The pointwise plate border differs from the geometric one by at most $\sqrt{2}a_w/s_p$ world units (Section 3.4). The border margin is therefore chosen to exceed this displacement, $\omega_{\text{border}} > \sqrt{2}a_w/s_p$. Every grid sample a river visits is border-safe, and the geometric domain is convex, so every point on every channel lies at least ω_{border} inside it and is owned pointwise by the river’s own plate. River-terrain blending extends a further ω_{river} from a channel (see Section 3.8), and the lake-safe margin accounts for the lake radius and this width. Keeping the blend band itself inside the pointwise plate would require $\omega_{\text{border}} \geq \sqrt{2}a_w/s_p + \omega_{\text{river}}$; with a smaller margin, the outer edge of the band can be clipped near a border, which affects the carved valley but never the channel itself. The margin therefore trades against the warp amplitude: a stronger warp produces more irregular plate borders but requires a wider margin, which leaves less of each plate available for rivers. In the implementation, $a_w = 0.01$ and $s_p = 1.5 \times 10^{-4}$, giving a maximum displacement of about 94 world units against a margin of $\omega_{\text{border}} = 100$.

3.7 Lazy Per-Plate River Cache

A plate’s river network is generated the first time a terrain sample falls on that plate and finds the cache absent. When a view or chunk request is made, the caches of the $(2\rho_a + 1)^2$ lattice plates around the plate owning the view center may also be built up front, for an active radius ρ_a , so that dense sampling does not stall on first contact. Caches are keyed by the plate’s integer lattice index. Each cache stores

the planning grid, the river nodes (with raw height, river height, and type), the segments (with endpoint river heights and Strahler order), and the accepted paths. Because rivers naturally should not cross tectonic boundaries—where plateaus and oceans generate within this system—each plate’s network can be generated and cached in isolation without triggering generation of neighboring plates.

For a terrain sample on plate i , let $\mathcal{R}_i$ be the cached set of river segments. Lakes are treated as disks of radius r_{lake} centered on local-minimum nodes, so the per-segment distance is

$$\delta_s(\mathbf{x}) = \min\left(\text{dist}(\mathbf{x}, s), \min_{\mathbf{p} \in L_s} \|\mathbf{x} - \mathbf{p}\| - r_{\text{lake}}\right), \quad (30)$$

where L_s is the set of lake endpoints of s (and the inner term is omitted when L_s is empty). The distance to the river network is computed exactly as $\delta_{\text{river}}(\mathbf{x}) = \min_{s \in \mathcal{R}_i} \delta_s(\mathbf{x})$ and is non-positive inside a lake. No spatial index or cached nearest-segment regions are used: every query scans the segments of the owning plate, at a cost of $O(|\mathcal{R}_i|)$ per sample.

Each segment s also provides a height $h_s(\mathbf{x})$, obtained by linearly interpolating its endpoint river heights at the clamped projection of $\mathbf{x}$ onto s . Within a margin m_{lake} of a lake, h_s is set to the lake height, and beyond that margin it fades back toward the interpolated value over a distance D_{lake} . The river height combines these per-segment heights:

$$H_{\text{river}}(\mathbf{x}) = \begin{cases} h_{\text{lake}}, & \text{in a lake,} \\ \text{mean}_{s: d_s=0} h_s(\mathbf{x}), & \text{on a channel,} \\ \frac{\sum_s w_s h_s(\mathbf{x})}{\sum_s w_s}, & \text{otherwise,} \end{cases} \quad (31)$$

with $d_s = \text{dist}(\mathbf{x}, s)$ and $w_s = (1 - S(d_s/D_h))^2/d_s^2$ over segments with $d_s < D_h$. If no segment lies within D_h , h_s of the nearest segment is used. Compared with taking the height of the single nearest segment, this inverse-distance blend removes height discontinuities where the nearest segment changes, such as around bends and confluences. On the channel of any segment s outside a lake surface, $H_{\text{river}} = h_s$. Segments never cross or overlap, so the only other segments at zero distance then share an endpoint with s and have the same height there. Because the lake fade only

pulls h_s toward the lower lake height as a channel approaches its lake, H_{river} never increases downstream along such a channel and is flat only on lake surfaces. Collinear overlaps cannot occur: a tracing step that touches the interior of an existing segment counts as a crossing and is rejected, and no source lies within $s_{\min}$ of an existing river.

The cached Strahler order can supply a river-size field, for example by taking the order of the nearest segment, and small deterministic noise offsets may be applied to the river distance, height, or size fields before final composition to make channels appear more curved. Neither affects the downhill guarantee, and neither is used in the height composition of the evaluated implementation.

3.8 On-Demand Final Terrain Composition

During chunk generation, plate weight fields are sampled, the distance $\delta_{\text{river}}(\mathbf{x})$ to the owning plate’s river network is computed, and river elevation $H_{\text{river}}(\mathbf{x})$ is evaluated by Equation (31). The river distance is converted into a normalized terrain blend weight before mixing the local terrain height with the river height:

$$\lambda_{\text{river}}(\mathbf{x}) = \left[\min \left(1, \frac{[\delta_{\text{river}}(\mathbf{x})]_+}{\omega_{\text{river}}} \right) \right]^{\alpha_{\text{river}}},$$

$$H_{\text{final}}(\mathbf{x}) = \lambda_{\text{river}}(\mathbf{x})H_{\text{pre}}(\mathbf{x}) + (1 - \lambda_{\text{river}}(\mathbf{x}))H_{\text{river}}(\mathbf{x}). \quad (32)$$

Here ω_{river} is the river blending width and α_{river} controls the blend falloff. Near the river, λ_{river} approaches zero and the interpolated river height dominates; away from the river, λ_{river} approaches one and the locally generated terrain is preserved. Because $\delta_{\text{river}} \leq 0$ inside a lake, $\lambda_{\text{river}} = 0$ there, producing a flat lake surface. On every channel $\lambda_{\text{river}} = 0$ and $H_{\text{final}} = H_{\text{river}}$, and every channel point is owned pointwise by the channel’s own plate (Section 3.6), so the downhill property of Section 3.7 carries over to the final terrain along every channel.

4 Experimental Results and Evaluation

A video demonstration for a finite, cube-shaped planet implementation of this system is available on

YouTube at youtube.com/watch?v=4Ai_13znvgg.

Code for the toy examples in Figure 1 is available at github.com/Davismk3/plate-local-river-generation.

Figure 1(a) and Table 1 show the output and measured performance of a single deterministic configuration of the flat infinite world, detailed below. Figures 1(b) and (c) illustrate the finite-world variants; no measurements are reported for them. All benchmark code and raw results accompany this manuscript on GitHub, and every statistic is computed directly from the released implementation.

The implementation is written in C++17 and has no third-party dependencies. It consists of a library for pointwise terrain evaluation, plate-local river construction, and lazy per-plate caching, together with a command-line heightmap exporter and a native interactive viewer. No game engine is used. The figure and metrics are produced by a separate exporter program that uses only the library’s public interface and links zlib for PNG output. It renders the terrain with a software rasterizer at $3\times$ supersampling and depth-tests river channels against the terrain. All code was compiled with Apple clang 17 at `-O3`.

4.1 Reference Configuration

A single exporter invocation produces the figure, a JSON summary, and a per-plate CSV file, so all reported measurements correspond to Figure 1(a). The run uses seed 1 and a square view two plate-cell widths across, centered on plate (1,0). The terrain is sampled at a resolution of 320×320 (102,400 samples). Ten plates own at least one sample in the view, and a river cache is built for each. River networks are generated on a 100×100 grid within each plate, with at most $K = 50$ river paths per plate and a source spacing of $s_{\min} = 5$ grid cells. The warp amplitude is $a_w = 0.01$ plate cells, so the pointwise and geometric plate borders differ by at most about 94 world units. The border margin is $\omega_{\text{border}} = 100$ world units, the river blending width is also 100 world units, the lake radius is 150 world units, and the planning attenuation is $a_{\text{relief}} = 0.35$. Channel lengths are normalized to plate-cell widths.

The sampled terrain contains 63.92% land and 36.08% ocean at a sea level of 0.35. Terrain heights range

from 0.200 to 1.435, with a median of 0.472 and a 95th percentile of 1.201.

The 10 plate caches contain 1,656 nodes and 1,450 segments. Of the 500 possible river paths, 222 are accepted (44.4%); each plate accepts 18–28 paths before it runs out of source candidates that are both border-safe and at least $s_{\min}$ from an existing river. The networks contain 222 source nodes, 200 sea outlets, 6 lakes, 11 confluences, and 206 connected components, and the highest Strahler order is 2. The total channel length across all plate caches is 49.766 plate-cell widths, or 4.977 per plate on average. Individual plates contain 108–235 segments, and their total channel lengths range from 3.852 to 7.734 plate-cell widths.

After clipping the river network to the meshed domain shown in Figure 1(a), 782 nodes and 655 segments remain. The visible network contains 130 source nodes, 81 sea outlets, 6 lakes, and 565 channel nodes, with a visible channel length of 22.582 plate-cell widths.

4.2 Downhill Verification

Downhill flow is verified at two levels. First, every stored river segment is tested using its adjusted end-point river heights. All 1,450 segments descend, as guaranteed by construction (Section 3.6).

Second, the final composed terrain H_{final} is sampled at 16 evenly spaced points along every segment (23,200 samples), and any downstream rise greater than 10^{-9} is counted. No segment rises at any sample. Every one of these samples is owned pointwise by its segment’s own plate, and no two segments overlap, confirming the guarantees of Sections 3.6–3.8.

Because the border margin equals the blending width rather than exceeding the warp displacement plus that width, the blend band around a channel is not guaranteed to stay within its plate. Sampling the band at $\pm \omega_{\text{river}}/2$ and $\pm \omega_{\text{river}}$ on either side of every channel sample finds 6 of 92,800 points (0.0065%) owned by a neighboring plate. At those points, the outer edge of the carved valley is clipped; the channels themselves are unaffected.

4.3 Memory and Timing

The cached nodes, segments, and paths of the 10 plates occupy 0.169 MiB (177,360 bytes), or approximately 122 bytes per river segment, at 64 bytes per node record and 40 bytes per segment record. Each cache also retains its planning grid, which occupies 4.10 MiB across the 10 plates. The grid is not needed for terrain queries and could be discarded after construction. Allocator and container overhead are excluded from both figures.

The measurements were collected on an Apple M4 Max system. Timings are the median of 5 repeats, each on a fresh world instance so that every cache is built cold; heights and networks were identical across repeats. Construction of the 10 plate caches requires 37.9 ms in total, with a mean of 3.81 ms and a median of 3.05 ms per cache, and individual caches take 2.33 to 10.07 ms. Sampling the 320×320 view with river queries requires 116 ms ($1.13 \mu\text{s}$ per sample), compared with 28.1 ms ($0.27 \mu\text{s}$ per sample) for pointwise terrain alone. The difference, about $0.85 \mu\text{s}$ per sample, is the cost of the exhaustive segment scan of Section 3.7. Rendering at $3 \times$ supersampling requires 0.161 s and PNG encoding 0.522 s. These timings are observational rather than controlled benchmarks and may vary with system load, whereas the terrain and river network statistics are deterministic.

Table 1. Metrics emitted by the same deterministic exporter invocation that produced Figure 1(a). Lengths are measured in plate-cell widths. Complete-network counts include all 10 plate caches intersecting the view; visible counts are clipped to the displayed mesh. Timings are medians of 5 cold repeats.

Metric	Value
Terrain samples	102,400
Visible plate caches	10
Land / ocean fraction	63.92 / 36.08%
Median / 95th-percentile height	0.472 / 1.201
Nodes / segments	1,656 / 1,450
Accepted / possible paths	222 / 500
Sources / confluences	222 / 11
Sea outlets / lakes	200 / 6
Connected components	206
Maximum Strahler order	2

continued on next column

Table 1 continued

Metric	Value
Stored-segment downhill violations	0
Final-terrain rising segments	0 / 1,450
Channel samples on other plates	0 / 23,200
Blend-band samples on other plates	6 / 92,800
Total / visible channel length	49.766 / 22.582
Visible nodes / segments	782 / 655
Network cache size	0.169 MiB
Planning-grid size	4.10 MiB
Cache build total / mean	37.9 / 3.81 ms
Sample with / without rivers	1.13 / 0.27 μ s
Render / PNG encode	0.161 / 0.522 s

5 Future Work

River queries currently scan every segment of the owning plate. A per-plate spatial index, such as a uniform bin grid or bounding-volume hierarchy over the segments, would reduce this cost for plates with dense networks without changing the exact distance.

The plate-local framework is not limited to rivers. The same bounded-domain pattern could generate roads, towns, or other features that guarantee a certain topology. Each application would replace downhill flow with its own constraints—for example slope and travel cost for roads, or water access and terrain suitability for towns—while retaining deterministic identifiers, lazy regional preprocessing, per-plate caches, and pointwise evaluation.

Most notably, integration with learned generators such as InfiniteDiffusion [13] offers a longer-term hybrid direction. Plate-local fields and river graphs could supply explicit structural constraints, while a learned model adds fine surface, coastline, and mountain detail.

6 Conclusion

We have presented an algorithm for lazily generating river networks that are terrain-aware, deterministic, and coherent, with downhill flow guaranteed along every river channel of the final terrain, and which can be tailored to flat infinite world-types or closed-shaped finite world-types such as planets.

References

- [1] F. Aurenhammer, “Voronoi diagrams—a survey of a fundamental geometric data structure,” *ACM Computing Surveys*, vol. 23, no. 3, pp. 345–405, 1991.
- [2] A. Okabe, B. Boots, K. Sugihara, and S. N. Chiu, *Spatial Tessellations: Concepts and Applications of Voronoi Diagrams*, 2nd ed. Wiley, 2000.
- [3] E. B. Saff and A. B. J. Kuijlaars, “Distributing many points on a sphere,” *The Mathematical Intelligencer*, vol. 19, pp. 5–11, 1997. doi: [10.1007/BF03024331](https://doi.org/10.1007/BF03024331).
- [4] R. Swinbank and R. James Purser, “Fibonacci grids: A novel approach to global modelling,” *Quarterly Journal of the Royal Meteorological Society*, vol. 132, pp. 1769–1793, 2006. doi: [10.1256/qj.05.227](https://doi.org/10.1256/qj.05.227).
- [5] K. Perlin, “An image synthesizer,” *ACM SIGGRAPH Computer Graphics*, vol. 19, no. 3, pp. 287–296, 1985.
- [6] S. Worley, “A cellular texture basis function,” in *Proceedings of SIGGRAPH 1996*, pp. 291–294, 1996.
- [7] I. E. Sutherland and G. W. Hodgman, “Reentrant polygon clipping,” *Communications of the ACM*, vol. 17, no. 1, pp. 32–42, 1974.
- [8] A. N. Strahler, “Quantitative analysis of watershed geomorphology,” *Eos, Transactions American Geophysical Union*, vol. 38, no. 6, pp. 913–920, 1957. doi: [10.1029/TR038i006p00913](https://doi.org/10.1029/TR038i006p00913).
- [9] B. Batbold and L. Ziegelmeier, “From chaos to continents: Voronoi-based procedural terrain generation with hydrology and 3D visualization,” in *42nd International Symposium on Computational Geometry (SoCG 2026)*, ser. Leibniz International Proceedings in Informatics (LIPIcs), vol. 367. Schloss Dagstuhl—Leibniz-Zentrum für Informatik, 2026, pp. 101:1–101:7. doi: [10.4230/LIPIcs.SoCG.2026.101](https://doi.org/10.4230/LIPIcs.SoCG.2026.101).
- [10] J.-D. Gènevaux, É. Galin, E. Guérin, A. Peytavie, and B. Benes, “Terrain generation using procedural models based on hydrology,” *ACM Transactions on Graphics*, vol. 32, no. 4, pp. 143:1–143:13, 2013. doi: [10.1145/2461912.2461996](https://doi.org/10.1145/2461912.2461996).
- [11] E. Derzapf, B. Ganster, M. Guthe, and R. Klein, “River networks for instant procedural planets,” *Computer Graphics Forum*, vol. 30, no. 7, pp. 2031–2040, 2011.
- [12] R. Fischer, J. Boeckers, and G. Zachmann, “Procedural generation of landscapes with water bodies using artificial drainage basins,” in *Advances in Computer Graphics (CGI 2022)*, Lecture Notes in Computer Science, vol. 13443. Springer, 2022, pp. 345–356. doi: [10.1007/978-3-031-23473-6_27](https://doi.org/10.1007/978-3-031-23473-6_27).
- [13] A. Goslin, “InfiniteDiffusion: Bridging learned fidelity and procedural utility for open-world terrain generation,” in *SIGGRAPH Conference Papers '26*. New York, NY, USA: ACM, 2026, 10 pages. doi: [10.1145/3799902.3811080](https://doi.org/10.1145/3799902.3811080).